

Asynchronous FIFO Design

This repo contains verilog code for an asynchronous FIFO.

Table of Contents

1. [Author](#)
2. [Introduction](#)
3. [Design Space Exploration and Design Strategies](#)
 - i. [Read and Write Operations](#)
 - a. [Operations](#)
 - b. [Full, empty and wrapping condition](#)
 - c. [Gray Code Counter](#)
 - ii. [Signals Definition](#)
 - iii. [Dividing System Into Modules](#)
 - a. [FIFO.v](#)
 - b. [FIFO_memory.v](#)
 - c. [two_ff_sync.v](#)
 - d. [rptr_empty](#)
 - e. [wptr_full.v](#)
4. [Testbench Case Implementation](#)
 - i. [Waveforms](#)
5. [Results](#)
6. [Conclusion](#)
7. [References](#)

Author

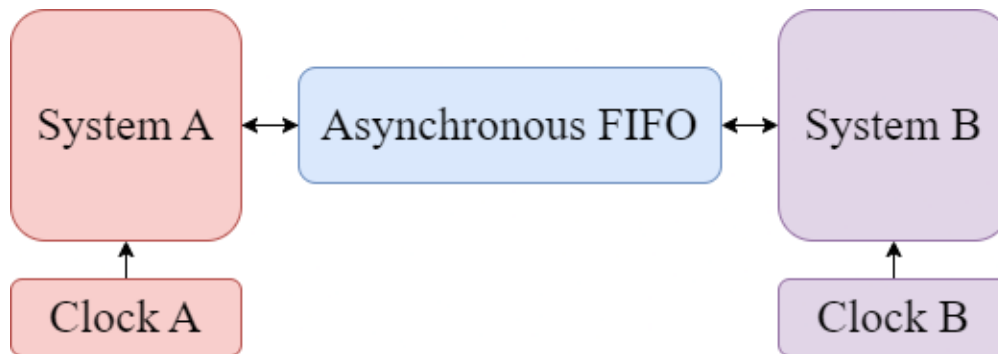
[UJJWAL CHAUDHARY](#), M. Tech. ESE 2023-25, IISc Bangalore

Introduction

- FIFO stands for "First-In, First-Out." It is a type of data structure or buffer in which the first data element added (the "first in") is the first one to be removed (the

"first out"). This structure is commonly used in scenarios where order of operations is important.

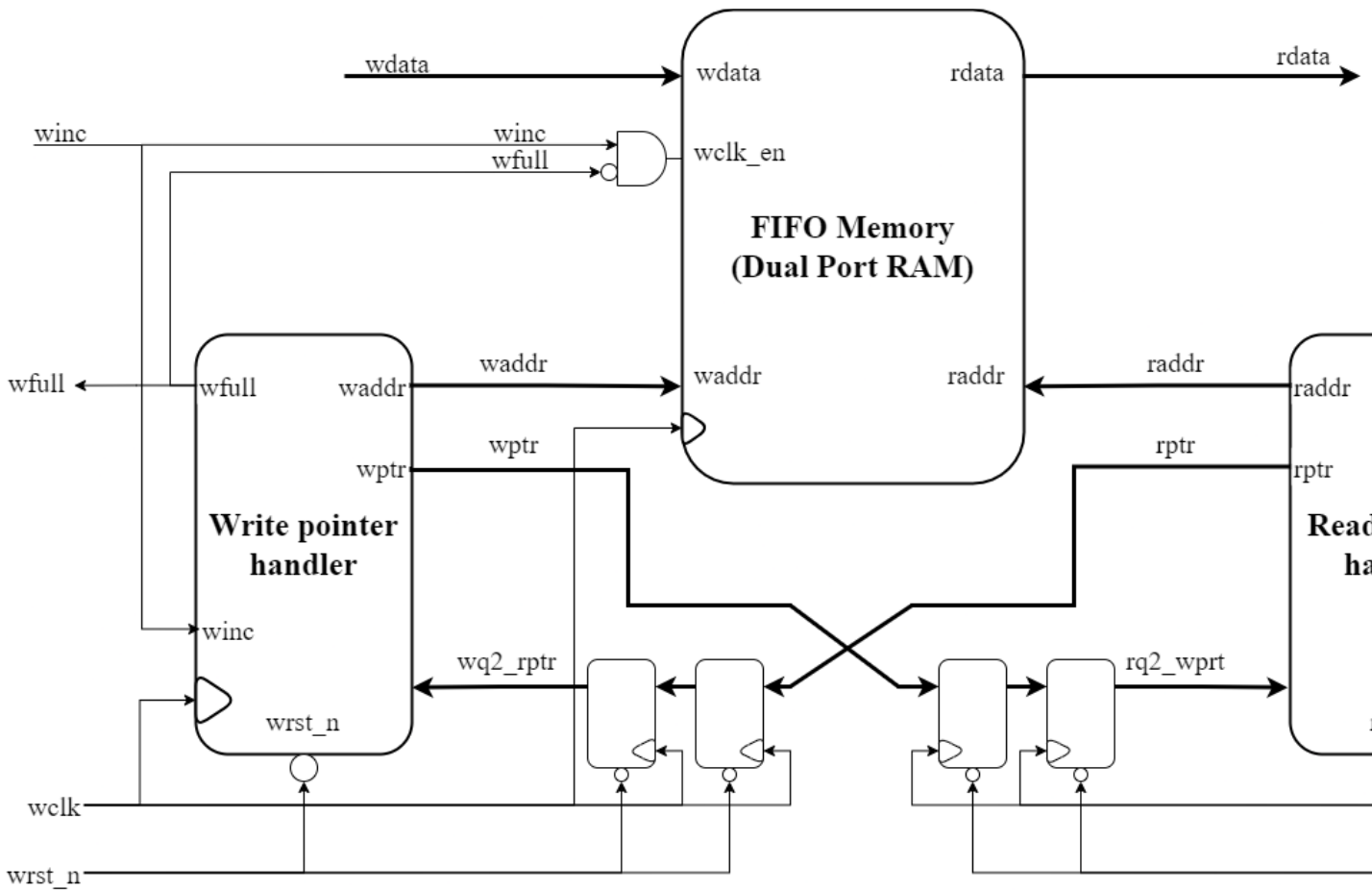
- Async FIFO, or Asynchronous FIFO, is a FIFO buffer where the read and write operations are controlled by independent clock domains. This means that the writing process and the reading process are driven by different clocks, which are not synchronized. Async FIFOs are used to safely transfer data between these asynchronous clock domains.



- Async FIFOs are used in various applications where data needs to be transferred between two parts of a system that operate on different clock frequencies. Some common use cases include:
 - Interfacing between different clock domains: For example, transferring data between a high-speed processing unit and a slower peripheral.
 - Communication between different modules: In a system-on-chip (SoC) where different modules might operate at different clock rates.
 - Buffering data: To handle variations in data flow rates between producer and consumer components in digital systems.
 - Bridging clock domains: In FPGA designs and other digital circuits where subsystems run at different clock speeds

Design Space Exploration and Design Strategies

The block diagram of async. FIFO that is implemented in this repo is given below. Thin lines represent single bit signal where as thick lines represent multi-bit signal.



Read and Write Operations

Operations

In an asynchronous FIFO, the read and write operations are managed by separate clock domains. The write pointer always points to the next word to be written. On a FIFO-write operation, the memory location pointed to by the write pointer is written, and then the write pointer is incremented to point to the next location to be written. Similarly, the read pointer always points to the current FIFO word to be read. On reset, both pointers are set to zero. When the first data word is written to the FIFO, the write pointer increments, the empty flag is cleared, and the read pointer, which is still addressing the contents of the first FIFO memory word, immediately drives that first valid word onto the FIFO data output port to be read by the receiver logic. The FIFO is empty when the read and write pointers are both equal, and it is full when the write pointer has wrapped around and caught up to the read pointer.

Full, empty and wrapping condition

The conditions for the FIFO to be full or empty are as follows:

- **Empty Condition:** The FIFO is empty when the read and write pointers are both equal. This condition happens when both pointers are reset to zero during a reset operation, or when the read pointer catches up to the write pointer, having read the last word from the FIFO.
- **Full Condition:** The FIFO is full when the write pointer has wrapped around and caught up to the read pointer. This means that the write pointer has incremented past the final FIFO address and has wrapped around to the beginning of the FIFO memory buffer.

To distinguish between the full and empty conditions when the pointers are equal, an extra bit is added to each pointer. This extra bit helps in identifying whether the pointers have wrapped around:

- **Wrapping Around Condition:** When the write pointer increments past the final FIFO address, it will increment the unused Most Significant Bit (MSB) while setting the rest of the bits back to zero. The same is done with the read pointer. If the MSBs of the two pointers are different, it means that the write pointer has wrapped one more time than the read pointer. If the MSBs of the two pointers are the same, it means that both pointers have wrapped the same number of times.

This design technique helps in accurately determining the full and empty conditions of the FIFO.

Gray code counter

Gray code counters are used in FIFO design because they only allow one bit to change for each clock transition. This characteristic eliminates the problem associated with trying to synchronize multiple changing signals on the same clock edge, which is crucial for reliable operation in asynchronous systems.

Signals Defination

Following is the list of signals used in the design with their defination:-

1. **wc1k:** Write clock signal.
2. **rc1k:** Read clock signal.
3. **wdata:** Write data bits.

4. **rdata**: Read data bits.
5. **wclk_en**: Write clock enable, this signal controls the write operation to the FIFO memory. Data must not be written if the FIFO memory is full ($wfull = 1$).
6. **wptr**: Write pointer (Gray).
7. **rpwr**: Read pointer (Gray).
8. **winc**: Write pointer increment. Controls the increment of the write pointer (**wptr**).
9. **rinc**: Read pointer increment. Controls the increment of the read pointer (**rpwr**).
10. **waddr**: Binary write pointer address. Location (address) of the FIFO memory to which data (**wdata**) to be written.
11. **raddr**: Binary read pointer address. Location (address) of the FIFO memory from where data (**rdata**) to be read.
12. **wfull**: FIFO full flag. Goes high if the FIFO memory is full.
13. **empty**: FIFO empty flag. Goes high if the FIFO memory is empty.
14. **wrst_n**: Active low asynchronous reset for the write pointer handler.
15. **rrst_n**: Active low asynchronous reset for the read pointer handler.
16. **w_rpwr**: Read pointer signal synchronized to the **wclk** domain via 2 flip-flop synchronized.
17. **r_wptr**: Write pointer signal synchronized to the **rclk** domain via 2 flip-flop synchronized.

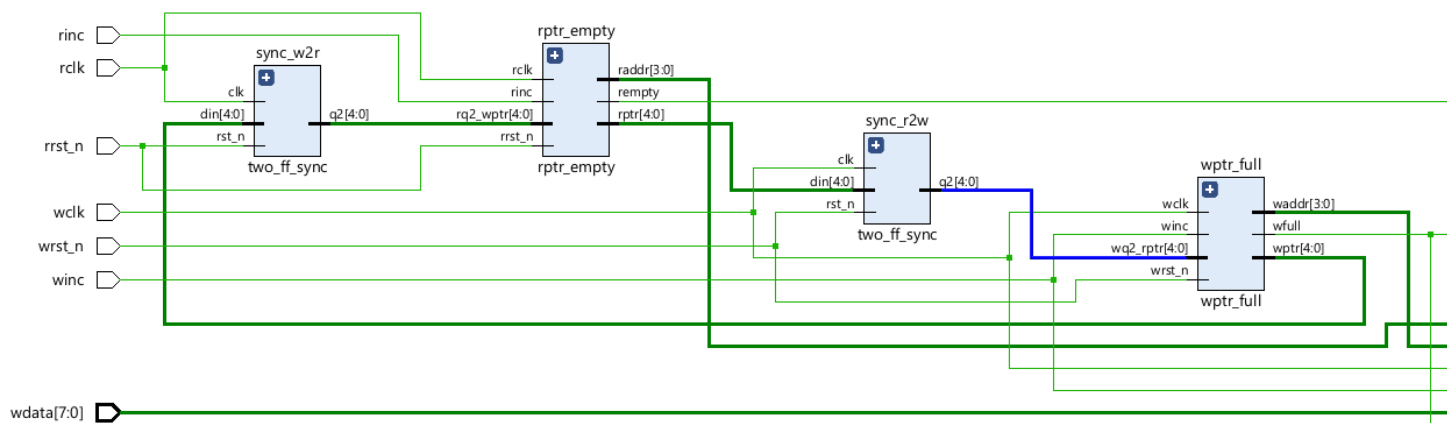
Dividing System Into Modules

For implementing this FIFO, I have divided the design into 5 modules:-

1. **FIFO.v**: The top-level wrapper module includes all clock domains and is used to instantiate all other FIFO modules. In a larger ASIC or FPGA design, this wrapper would likely be discarded to group the FIFO modules by clock domain for better synthesis and static timing analysis.
2. **FIFO_memory.v**: This module contains the buffer or the memory of the FIFO, which has both the clocks. This is a dual port RAM.
3. **two_ff_sync.v**: This module consists of two flip-flops that are connected to each other to form a 2 flip-flop synchronizer. This module will have two instances, one for write to read clock pointer synchronization and other for read to write clock pointer synchronization.
4. **rpwr_empty.v**: This module consists of the logic for the Read pointer handler. It is completely synchronized by read clock and consists of the logic for generation of FIFO empty signal.
5. **wptr_empty.v**: This module consists of the logic for the Write pointer handler. It is completely synchronized by write clock and consists of the logic for generation of FIFO full signal.

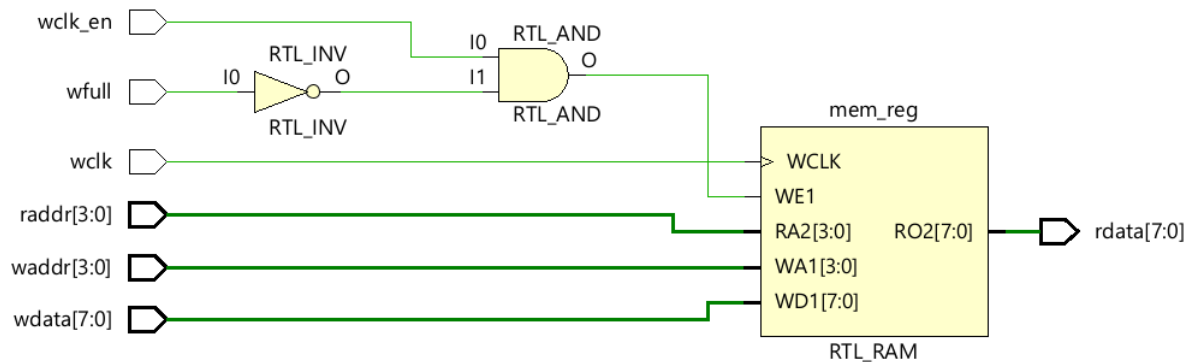
FIFO.v

[./Verilog_code/FIFO.v](#) is the code of this module. This module is a FIFO implementation with configurable data and address sizes. It consists of a memory module, read and write pointer handling modules, and read and write pointer synchronization modules. The read and write pointers are synchronized to the respective clock domains, and the read and write pointers are checked for empty and full conditions, respectively. The FIFO memory module stores the data and handles the read and write operations. The RTL schematics of this module is given below.



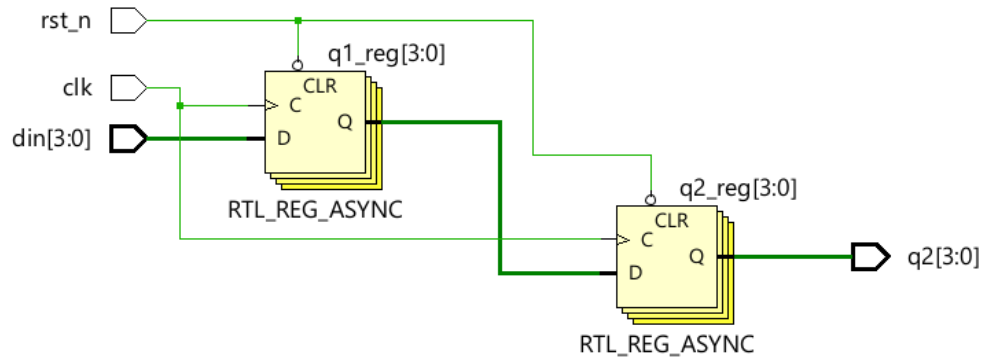
FIFO_memory.v

[./Verilog_code/FIFO_memory.v](#) is the code of this module. The module has a memory array (`mem`) with a depth of $2^{\text{ADDR_SIZE}}$. The read and write addresses are used to access the memory array. The write clock enable (`wclk_en`) and write full (`wfull`) signals are used to control the writing process. The write data is stored in the memory array on the rising edge of the write clock (`wclk`). The RTL schematics of this module is given below.



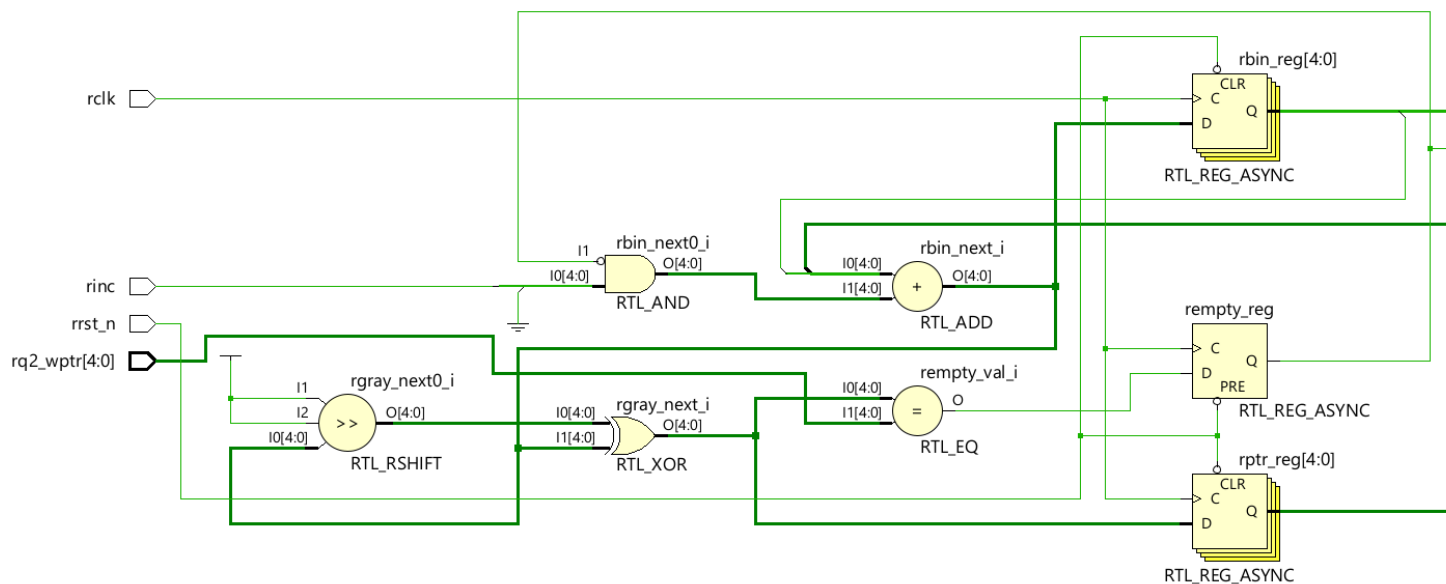
two_ff_sync.v

[./Verilog_code/two_ff_sync.v](#) is the code of this module. The module has two flip-flops, q1 and q2, which store the input data (din) of size SIZE. On each clock cycle, the data is shifted from q1 to q2, and new data is loaded into q1. The reset signal (rst_n) is active low, meaning the FIFO is reset when rst_n is low. The RTL schematics of this module is given below.



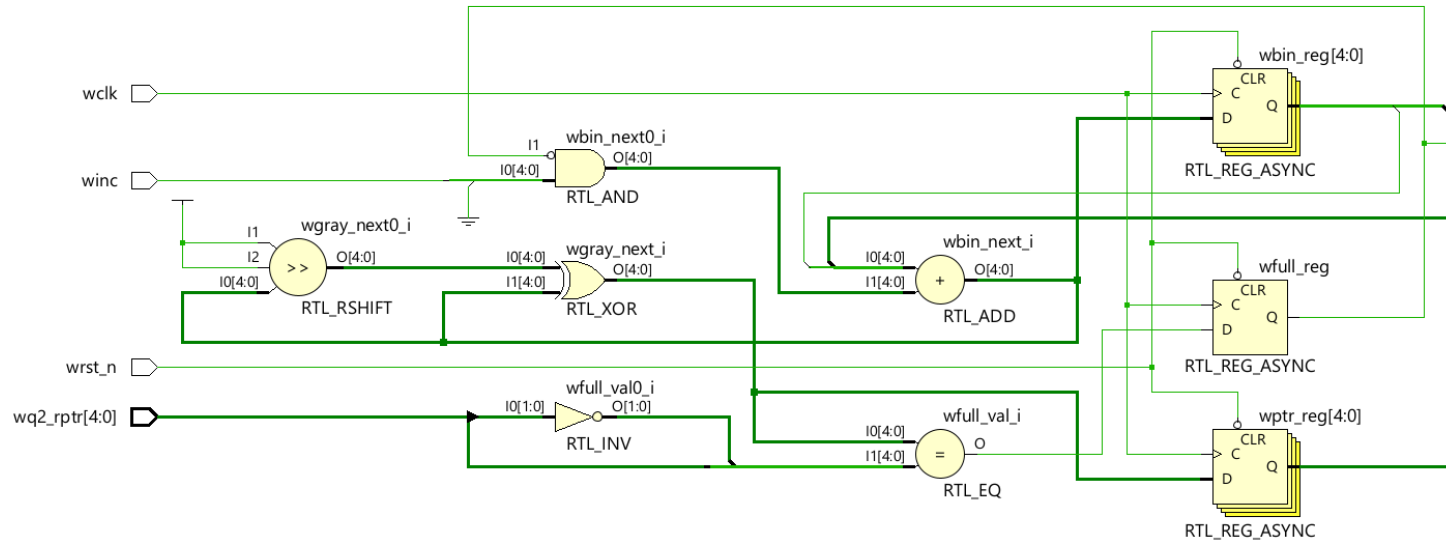
rptr_empty.v

[./Verilog_code/rprt_empty.v](#) is the code of this module. The module implements a read pointer for a FIFO with an empty flag. The read pointer is implemented in grey code to avoid glitches when transitioning clock domains. The read pointer is incremented based on the read increment signal and the empty flag. The empty flag is set when the read pointer is equal to the write pointer, indicating that the FIFO is empty. The read pointer and empty flag are updated on each clock cycle, and the read address is calculated from the read pointer. The RTL schematics of this module is given below.



wp_ptr_full.v

[./Verilog_code/wprrt_full.v](#) is the code of this module. The module implements a write pointer for a FIFO with a full flag. The write pointer is implemented in gray code to avoid glitches when transitioning between clock domains. The write pointer is incremented based on the write increment signal and the full flag. The full flag is set when the write pointer is equal to the read pointer, indicating that the FIFO is full. The write pointer and full flag are updated on each clock cycle, and the write address is calculated from the write pointer. The RTL schematics of this module is given below.



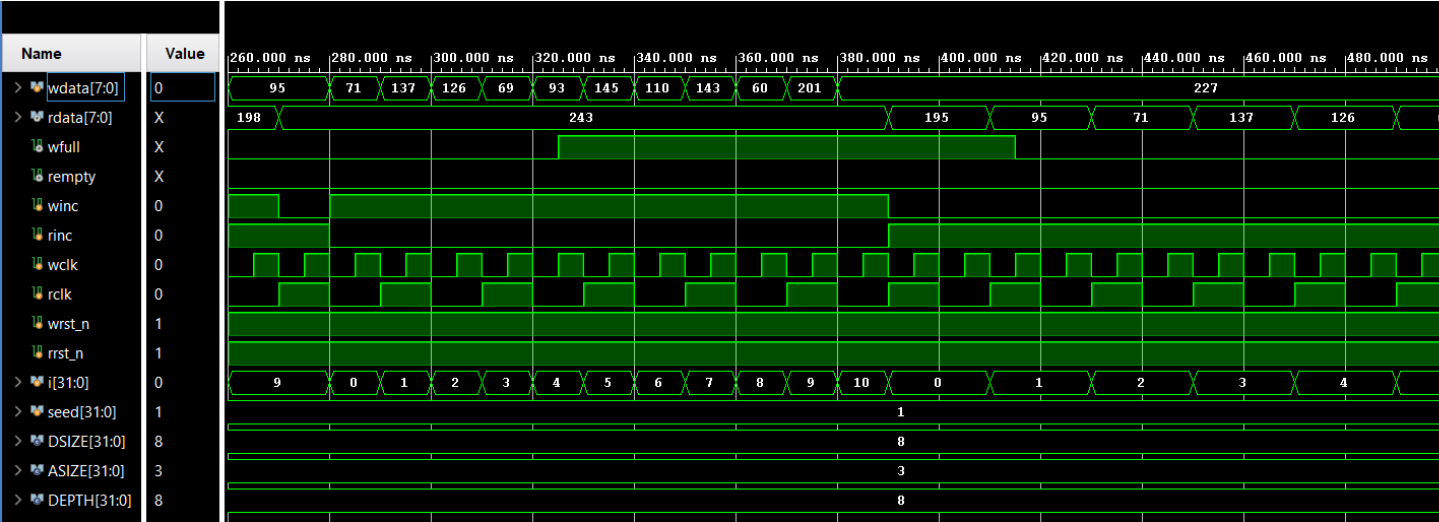
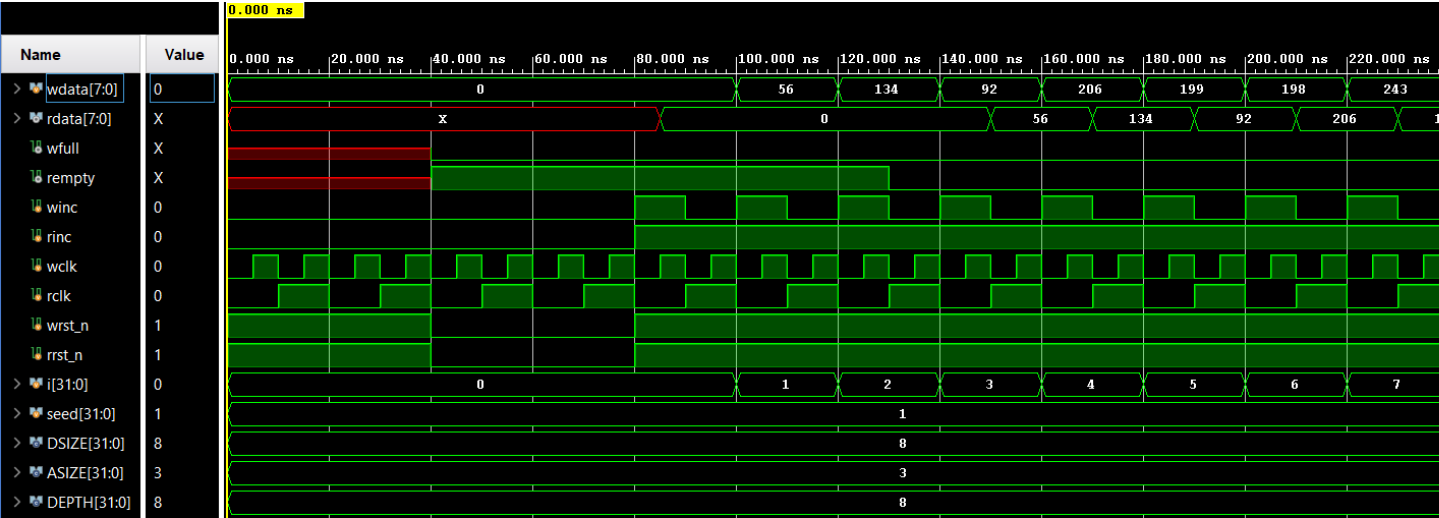
Testbench Case Implementation

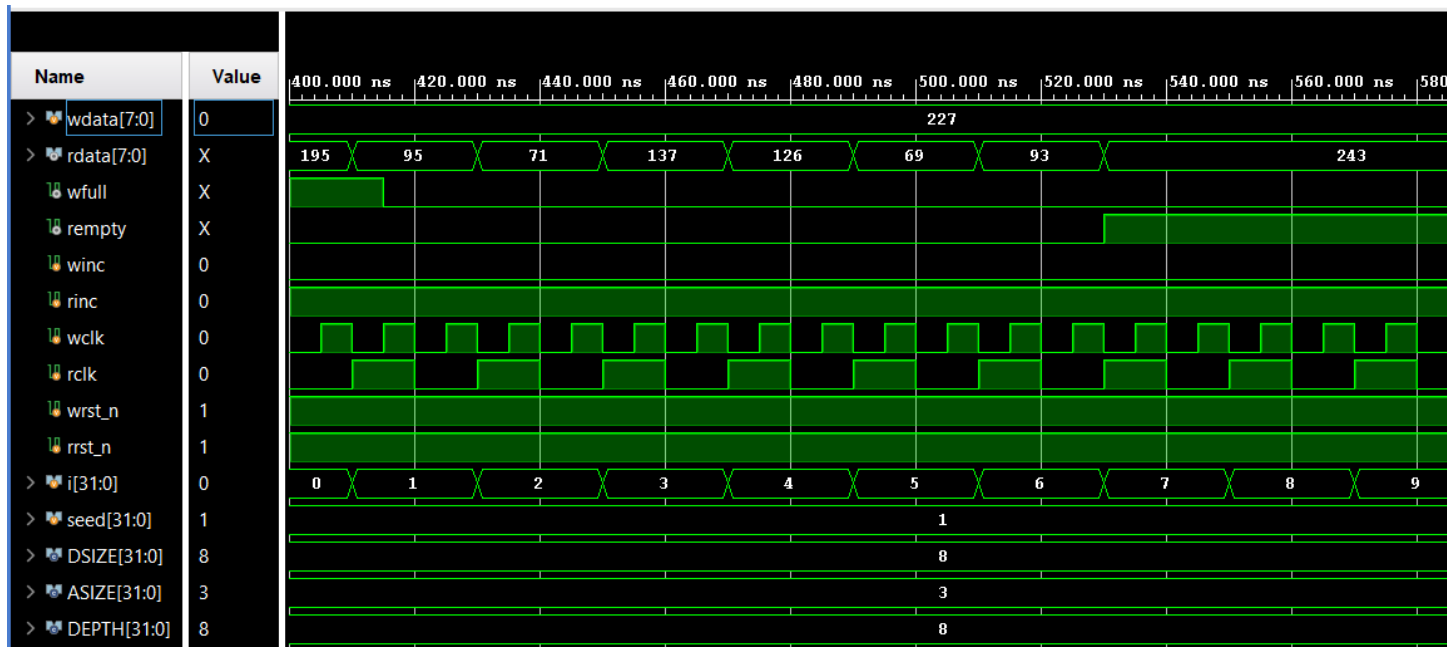
[./Verilog_code/FIFO_tb.v](#) is the code of this module. The testbench for the FIFO module generates random data and writes it to the FIFO, then reads it back and compares the results. The testbench includes three test cases:-

1. Write data and read it back.
2. Write data to make the FIFO full and try to write more data.
3. Read data from an empty FIFO and try to read more data.

The testbench uses clock signals for writing and reading, and includes reset signals to initialize the FIFO. The testbench finishes after running the test cases.

Waveforms





Results

The asynchronous FIFO design was tested using a testbench. The following key results were observed:-

1. **Correct Data Storage and Retrieval:** The FIFO correctly stored data when written to and retrieved the exact same data when read from. This was validated across multiple test cases with varying data patterns.
2. **Full and Empty Conditions:** The FIFO accurately indicated full and empty conditions. When the FIFO was full, additional write operations were correctly prevented, and when the FIFO was empty, additional read operations were correctly halted.

Conclusion

The design and implementation of the asynchronous FIFO were successful, demonstrating reliable data storage and retrieval between asynchronous clock domains. The use of gray code counters ensured proper synchronization, and the module's behavior in full and empty conditions was as expected. The testbench validated the FIFO's functionality across different scenarios, proving the design's correctness and efficiency.

While simulations confirmed the functional aspects of the design, it is important to note that metastability issues cannot be fully tested through simulations alone. Metastability is a physical phenomenon that occurs in actual hardware, and its mitigation relies on proper design techniques like the use of synchronizers and careful consideration of setup and hold times.

Overall, the asynchronous FIFO design is well-suited for applications requiring data transfer between different clock domains, ensuring data integrity and synchronization. Future work could involve implementing the design on actual hardware to observe real-world behavior and further testing under varied clock frequencies and data patterns to ensure robust performance.

References

1. [Sunburst Design: Simulation and Synthesis Techniques for Asynchronous FIFO Design](#)
2. [VLSI verify Blog - Asynchronous FIFO](#)